Remote Attestation 구현 과정의 코드 분석

개요

이 문서는 Intel SGX SDK의 Remote Attestation 구현 과정을 실제 코드 흐름을 따라가면서 상세히 분석합니다. sgx_create_report() 함수부터 시작하여 EREPORT 명령어 래퍼, AESM Service와의 IPC 통신, Quoting Enclave의 Quote 생성 및 EPID 서명, 그리고 Intel Attestation Service와의 네트워크 통신까지 전체 과정을 코드 레벨에서 추적합니다.

1. sgx_create_report() 함수에서 EREPORT 명령어 래퍼까지의 코드 추적

1.1 sgx_create_report() 함수 구현

파일 위치: `sdk/selib/sgx_create_report.cpp`

```
sgx_status_t sgx_create_report(const sgx_target_info_t *target_info,
const sgx_report_data_t *report_data,
sgx_report_t *report)
{
static_assert(sizeof(*target_info) == 512, "sgx_target_info_t");
static_assert(sizeof(*report_data) == 64, "sgx_report_data_t");
static_assert(sizeof(*report) == 432, "sgx_report_t");

// 정렬된 임시 구조체들 생성
```

```
alignas(REPORT_DATA_ALIGN_SIZE) sgx_report_data_t tmp_report_data;

__memset((void *)&tmp_report_data, 0, sizeof(sgx_report_data_t));

alignas(TARGET_INFO_ALIGN_SIZE) sgx_target_info_t tmp_target_info;

__memset((void *)&tmp_target_info, 0, sizeof(sgx_target_info_t));

alignas(REPORT_ALIGN_SIZE)sgx_report_t tmp_report;

__memset((void *)&tmp_report, 0, sizeof(sgx_report_t));



// 매개변수 검증 및 복사

if(target_info) {

if (!sgx_is_within_enclave(target_info, sizeof(*target_info)))

return SGX_ERROR_INVALID_PARAMETER;

tmp_target_info = *target_info;

}

if(report_data) {

if(!sgx_is_within_enclave(report_data, sizeof(*report_data)))

return SGX_ERROR_INVALID_PARAMETER;

tmp_report_data = *report_data;

}

if(!report || !sgx_is_within_enclave(report, sizeof(*report))) {

return SGX_ERROR_INVALID_PARAMETER;

}



// EREPORT 명령어 실행
```

```
auto failed = do_ereport(&tmp_target_info, &tmp_report_data, &tmp_report);

// 결과를 사용자 버퍼로 복사 (memcpy 대신 루프 사용)

if (!failed) {

static_assert(sizeof(*report) % sizeof(uint64_t) == 0,

"sizeof(sgx_report_t) should be multiple of 8");

for(size_t i = 0; i < sizeof(*report)/sizeof(uint64_t); i++) {

((uint64_t*)report)[i] = ((uint64_t*)&tmp_report)[i];

}

}



return failed ? SGX_ERROR_UNEXPECTED : SGX_SUCCESS;

}
```

핵심 분석 포인트:

- 매개변수 정렬 요구사항: TARGET_INFO는 512바이트, REPORT_DATA는 64바이트 정렬
- Enclave 내부 메모리 검증: `sgx_is_within_enclave()` 함수로 포인터 유효성 검사
- 임시 버퍼 사용: 정렬 요구사항을 만족하는 스택 변수 사용
- memcpy 회피: Enclave 초기화 중에는 memcpy 사용 불가하므로 수동 루프 복사

1.2 do_ereport() 어셈블리 래퍼 구현

파일 위치: `sdk/trts/linux/trts_pic.S`

```
/*

* extern "C" int do_ereport(sgx_target_info_t *target_info,
```

```
* sgx_report_data_t *report_data,

* sgx_report_t *report);

* EREPORT: rbx - TARGETINFO 주소

* rcx - REPORTDATA 주소

* rdx - REPORT 출력 주소

* 반환값: 0 = 성공, 0이 아님 = 실패

*/

.global Lereport_inst

DECLARE_LOCAL_FUNC do_ereport

SE_PROLOG // 스택 프레임 설정

mov $SE_EREPORT, %xax // EREPORT leaf 번호 설정

clc // Carry flag 클리어

Lereport_inst:

ENCLU // EREPORT 명령어 실행

setc %al // Carry flag를 AL 레지스터로 복사

SE_EPILOG // 스택 프레임 정리

END_FUNC
```

핵심 분석 포인트:

- **SE_EREPORT**: EREPORT 명령어의 leaf 번호 (0x00000000)
- **레지스터 규약**: RBX=target_info, RCX=report_data, RDX=report 출력
- **에러 처리**: ENCLU 실행 후 Carry flag로 성공/실패 판단
- **ENCLU 매크로**: 실제 SGX 하드웨어 명령어 `ENCLU` 실행

1.3 Report 구조체 생성 과정

EREPORT 명령어는 하드웨어에서 다음과 같은 정보를 포함한 Report를 생성합니다:

```
typedef struct _sgx_report_t {
sgx_report_body_t body; // Report 본문
sgx_key_id_t key_id; // 키 식별자
sgx_mac_t mac; // MAC (Message Authentication Code)
} sgx_report_t;
```

```
typedef struct _sgx_report_body_t {
sgx_cpu_svn_t cpu_svn; // CPU 보안 버전 번호
sgx_misc_select_t misc_select; // 기타 선택 사항
uint8_t reserved1[28]; // 예약된 공간
sgx_attributes_t attributes; // Enclave 속성
sgx_measurement_t mr_enclave; // Enclave 측정값 (SHA256)
uint8_t reserved2[32]; // 예약된 공간
sgx_measurement_t mr_signer; // 서명자 측정값 (SHA256)
uint8_t reserved3[96]; // 예약된 공간
sgx_prod_id_t isv_prod_id; // ISV 제품 ID
sgx_isv_svn_t isv_svn; // ISV 보안 버전 번호
uint8_t reserved4[60]; // 예약된 공간
sgx_report_data_t report_data; // 사용자 데이터 (64바이트)
```

```
} sgx_report_body_t;
```

2. AESM Service와의 IPC 메커니즘 및 프로토콜 구현

2.1 AESM Service 아키텍처

AESM (Architecture Enclave Service Manager) Service는 다음과 같은 구조로 구성됩니다:

파일 위치: `psw/ae/aesm_service/source/core/`

- **CAESMServer**: 메인 서버 클래스
- **AESMWorkerThread**: 작업자 스레드 관리
- **AESMQueueManager**: 요청 큐 관리
- **IPC 인터페이스**: Unix Domain Socket 기반 통신

2.2 Quote 요청 IPC 구현

파일 위치: `psw/ae/aesm_service/source/core/ipc/AEGetQuoteRequest.cpp`

```
class AEGetQuoteRequest : public IAERequest {
private:
aesm::message::Request::GetQuoteRequest* m_request;

public:
// 생성자: Protobuf 메시지로부터 요청 객체 생성
AEGetQuoteRequest(uint32_t reportLength, const uint8_t* report,
uint32_t quoteType,
uint32_t spidLength, const uint8_t* spid,
```

```
uint32_t  nonceLength, const  uint8_t*  nonce,
uint32_t  sig_rlLength, const  uint8_t*  sig_rl,
uint32_t  bufferSize,
bool  qe_report,
uint32_t  timeout) : m_request(NULL)
{
m_request = new  aesm::message::Request::GetQuoteRequest();
// 매개변수들을 Protobuf 메시지에 설정
if (reportLength !=0 && report != NULL)
m_request->set_report(report, reportLength);
if (spidLength != 0 && spid != NULL)
m_request->set_spid(spid, spidLength);
if (nonceLength != 0 && nonce != NULL)
m_request->set_nonce(nonce, nonceLength);
if (sig_rlLength != 0 && sig_rl != NULL)
m_request->set_sig_rl(sig_rl, sig_rlLength);
m_request->set_quote_type(quoteType);
m_request->set_buf_size(bufferSize);
m_request->set_qe_report(qe_report);
m_request->set_timeout(timeout);
}


// 메시지 직렬화
```

```
AEMessage*  serialize() {
AEMessage *ae_msg = NULL;
aesm::message::Request msg;
if (check()) {
aesm::message::Request::GetQuoteRequest* mutableReq =
msg.mutable_getquotereq();
mutableReq->CopyFrom(*m_request);


if (msg.ByteSize() <= INT_MAX) {
ae_msg = new AEMessage;
ae_msg->size = (unsigned  int)msg.ByteSize();
ae_msg->data = new  char[ae_msg->size];
msg.SerializeToArray(ae_msg->data, ae_msg->size);
}
}
return ae_msg;
}


// 요청 실행
IAEResponse*  execute(IAESMLogic*  aesmLogic) {
aesm_error_t result = AESM_UNEXPECTED_ERROR;
uint32_t qe_report_length = 0;
uint8_t* qe_report = NULL;
```

```
uint32_t quote_length = 0;
uint8_t* quote = NULL;

if (check()) {
// Protobuf 메시지에서 데이터 추출
uint32_t report_length = 0;
uint8_t* report = NULL;
uint32_t spid_length = 0;
uint8_t* spid = NULL;
uint32_t nonce_length = 0;
uint8_t* nonce = NULL;
uint32_t sig_rl_length = 0;
uint8_t* sig_rl = NULL;

if (m_request->has_report()) {
report_length = (unsigned  int)m_request->report().size();
report = (uint8_t*)const_cast<char *>(m_request->report().data());
}
// ... 다른 필드들도 유사하게 추출

quote_length = (uint32_t)m_request->buf_size();

// AESM 로직 호출하여 Quote 생성
```

```
result = aesmLogic->getQuote(report_length, report,
(uint32_t)m_request->quote_type(),
spid_length, spid,
nonce_length, nonce,
sig_rl_length, sig_rl,
quote_length, &quote,
m_request->qe_report(), &qe_report_length, &qe_report);
}
// 응답 객체 생성
AEGetQuoteResponse* response = new  AEGetQuoteResponse(result,
quote_length, quote,
qe_report_length, qe_report);
// 메모리 정리
if (quote) delete[] quote;
if (qe_report) delete[] qe_report;
return response;
}
};
```

2.3 Unix Domain Socket 통신 구현

파일 위치: `psw/ae/aesm_service/source/core/ipc/UnixCommunicationSocket.cpp`

```
class  UnixCommunicationSocket : public  ICommunicationSocket {
```

```
private:
int mSocket;
struct sockaddr_un mSocketAddress;


public:
// 소켓 연결
uae_oal_api_t connect() {
mSocket = socket(AF_UNIX, SOCK_STREAM, 0);
if (mSocket == -1) {
return UAE_OAL_ERROR_UNEXPECTED;
}


memset(&mSocketAddress, 0, sizeof(mSocketAddress));
mSocketAddress.sun_family = AF_UNIX;
strncpy(mSocketAddress.sun_path, AESM_SOCKET_PATH,
sizeof(mSocketAddress.sun_path) - 1);


if (::connect(mSocket, (struct sockaddr*)&mSocketAddress,
sizeof(mSocketAddress)) == -1) {
close(mSocket);
return UAE_OAL_ERROR_UNEXPECTED;
}
```

```
return UAE_OAL_SUCCESS;
}

// 데이터 송신
uae_oal_api_t writeRaw(const char* data, uint32_t length) {
ssize_t bytes_sent = send(mSocket, data, length, 0);
if (bytes_sent != (ssize_t)length) {
return UAE_OAL_ERROR_UNEXPECTED;
}
return UAE_OAL_SUCCESS;
}

// 데이터 수신
uae_oal_api_t readRaw(char* data, uint32_t length) {
ssize_t bytes_received = recv(mSocket, data, length, MSG_WAITALL);
if (bytes_received != (ssize_t)length) {
return UAE_OAL_ERROR_UNEXPECTED;
}
return UAE_OAL_SUCCESS;
}
};
```

2.4 Protobuf 직렬화 메커니즘

파일 위치: `psw/ae/aesm_service/source/core/ipc/messages.proto`

```
syntax = "proto2";
package aesm.message;

message Request {
message GetQuoteRequest {
optional bytes report = 1;
optional uint32 quote_type = 2;
optional bytes spid = 3;
optional bytes nonce = 4;
optional bytes sig_rl = 5;
optional uint32 buf_size = 6;
optional bool qe_report = 7;
optional uint32 timeout = 8;
}
optional GetQuoteRequest getquotereq = 1;
// ... 다른 요청 타입들
}

message Response {
message GetQuoteResponse {
optional uint32 errorcode = 1;
```

```
optional bytes quote = 2;

optional bytes qe_report = 3;

}

optional GetQuoteResponse getquoteres = 1;

// ... 다른 응답 타입들

}
```

3. Quoting Enclave의 Quote 생성 알고리즘 및 EPID 서명 과정

3.1 Quoting Enclave 인터페이스

파일 위치: `psw/ae/qe/quoting_enclave.edl`

```
enclave {

include "sgx_report.h"

include "sgx_quote.h"

from "sgx_tstdc.edl" import *;

trusted {

public uint32_t get_quote([size = blob_size, in, out] uint8_t *p_blob,

uint32_t blob_size,

[in] const sgx_report_t *p_report,

sgx_quote_sign_type_t quote_type,

[in] const sgx_spid_t *p_spid,

[in] const sgx_quote_nonce_t *p_nonce,
```

```
[user_check] const uint8_t *p_sig_rl,

uint32_t sig_rl_size,

[out] sgx_report_t *qe_report,

[user_check] uint8_t *p_quote,

uint32_t quote_size,

sgx_isv_svn_t pce_isvnsvn);

};

};
```

3.2 get_quote() 함수 구현 분석

파일 위치: `psw/ae/qe/quoting_enclave.cpp`

```
uint32_t  get_quote(

uint8_t  *p_blob, // EPID 블롭

uint32_t  blob_size,

const  sgx_report_t  *p_enclave_report, // 대상 Enclave의 Report

sgx_quote_sign_type_t  quote_type, // 서명 타입 (Linkable/Unlinkable)

const  sgx_spid_t  *p_spid, // Service Provider ID

const  sgx_quote_nonce_t  *p_nonce, // Nonce (옵션)

const  uint8_t  *emp_sig_rl, // Signature Revocation List

uint32_t  sig_rl_size,

sgx_report_t  *p_qe_report, // QE Report 출력

uint8_t  *emp_quote, // Quote 출력 버퍼
```

```
uint32_t  quote_size,

sgx_isv_svn_t  pce_isvsvn) // PCE ISV SVN

{

ae_error_t ret = AE_SUCCESS;

EpidStatus epid_ret = kEpidNoErr;

MemberCtx *p_epid_context = NULL;

sgx_quote_t quote_body;

uint8_t is_resealed = 0;

sgx_basename_t basename = {{0}};

uint64_t sign_size = 0;

sgx_status_t se_ret = SGX_SUCCESS;

sgx_report_t qe_report;

uint64_t required_buffer_size = 0;

se_sig_rl_t sig_rl_header;

se_plaintext_epid_data_sdk_t plaintext;

sgx_ec256_signature_t ec_signature;

sgx_cpu_svn_t cpusvn;



// 1. 매개변수 검증

if((NULL == p_blob) || (NULL == p_enclave_report) ||

(NULL == p_spid) || (NULL == emp_quote) || (!quote_size)) {

return QE_PARAMETER_ERROR;

}
```

```
if(SGX_TRUSTED_EPID_BLOB_SIZE_SDK != blob_size) {
return QE_PARAMETER_ERROR;
}
if(SGX_LINKABLE_SIGNATURE != quote_type &&
SGX_UNLINKABLE_SIGNATURE != quote_type) {
return QE_PARAMETER_ERROR;
}

// 2. 메모리 경계 검사
if(!sgx_is_outside_enclave(emp_sig_rl, sig_rl_size)) {
return QE_PARAMETER_ERROR;
}
if(!sgx_is_outside_enclave(emp_quote, quote_size)) {
return QE_PARAMETER_ERROR;
}
if(!sgx_is_within_enclave(p_blob, blob_size)) {
return QE_PARAMETER_ERROR;
}

// 3. 입력 Report 검증
if(SGX_SUCCESS != sgx_verify_report(p_enclave_report)) {
return QE_PARAMETER_ERROR;
}
```

```
// 4. EPID 블롭 검증 및 컨텍스트 생성
ret = random_stack_advance(verify_blob_internal, p_blob,
blob_size, &is_resealed, TRUE, plaintext,
&p_epid_context, &cpusvn);
if(AE_SUCCESS != ret) {
goto CLEANUP;
}

// 5. Signature Revocation List 처리
if(emp_sig_rl) {
uint64_t temp_size = 0;
uint64_t n2 = 0;

memcpy(&sig_rl_header, emp_sig_rl, sizeof(sig_rl_header));
// SigRL 헤더 검증
if(sig_rl_header.protocol_version != SE_EPID_SIG_RL_VERSION) {
ret = QE_PARAMETER_ERROR;
goto CLEANUP;
}
if(sig_rl_header.epid_identifier != SE_EPID_SIG_RL_ID) {
ret = QE_PARAMETER_ERROR;
goto CLEANUP;
```

```
}

// EPID 그룹 ID 검증

if(memcmp(&sig_rl_header.sig_rl.gid, &plaintext.epid_group_cert.gid,

sizeof(sig_rl_header.sig_rl.gid))) {

ret = QE_PARAMETER_ERROR;

goto CLEANUP;

}

// SigRL 크기 계산

temp_size = se_get_sig_rl_size(&sig_rl_header);

if(temp_size != sig_rl_size) {

ret = QE_PARAMETER_ERROR;

goto CLEANUP;

}



// ECDSA 서명 추출

memcpy(ec_signature.x,

emp_sig_rl + sig_rl_size - (SE_ECDSA_SIGN_SIZE * 2),

sizeof(ec_signature.x));

SWAP_ENDIAN_32B(ec_signature.x);

memcpy(ec_signature.y,

emp_sig_rl + sig_rl_size - (SE_ECDSA_SIGN_SIZE * 1),

sizeof(ec_signature.y));

SWAP_ENDIAN_32B(ec_signature.y);
```

```
// 서명 크기 계산
n2 = SWAP_4BYTES(sig_rl_header.sig_rl.n2);
temp_size = sizeof(EpidSignature) - sizeof(NrProof) + n2 * sizeof(NrProof);
if(temp_size > UINT32_MAX) {
ret = QE_PARAMETER_ERROR;
goto CLEANUP;
}
sign_size = temp_size;
} else {
// SigRL이 없는 경우 기본 서명 크기
sign_size = sizeof(BasicSignature) + sizeof(uint32_t) + sizeof(uint32_t);
}


// 6. Basename 생성
se_static_assert(sizeof(basename) > sizeof(*p_spid));
memcpy(&basename, p_spid, sizeof(*p_spid));
if(SGX_UNLINKABLE_SIGNATURE == quote_type) {
// Unlinkable 서명의 경우 랜덤 값 추가
uint8_t *p = (uint8_t *)&basename + sizeof(*p_spid);
se_ret = sgx_read_rand(p, sizeof(basename) - sizeof(*p_spid));
if(SGX_SUCCESS != se_ret) {
ret = QE_UNEXPECTED_ERROR;
```

```
goto CLEANUP;

}

}


// 7. EPID 컨텍스트에 Basename 등록

epid_ret = EpidRegisterBasename(p_epid_context, (uint8_t *)&basename,

sizeof(basename));

if(kEpidNoErr != epid_ret) {

ret = QE_UNEXPECTED_ERROR;

goto CLEANUP;

}


// 8. Quote 버퍼 크기 검증

required_buffer_size = SE_QUOTE_LENGTH_WITHOUT_SIG + sign_size;

if(quote_size < required_buffer_size) {

ret = QE_PARAMETER_ERROR;

goto CLEANUP;

}


// 9. Quote 본문 구성

memset_verw(emp_quote, 0, quote_size);

quote_body.version = QE_QUOTE_VERSION;

quote_body.sign_type = (uint16_t)quote_type;
```

```
quote_body.pce_svn = pce_isvsvn;

quote_body.xeid = plaintext.xeid;

// EPID 그룹 ID 설정 (엔디안 변환)

((uint8_t *)(&quote_body.epid_group_id))[0] = plaintext.epid_group_cert.gid.data

((uint8_t *)(&quote_body.epid_group_id))[1] = plaintext.epid_group_cert.gid.data

((uint8_t *)(&quote_body.epid_group_id))[2] = plaintext.epid_group_cert.gid.data

((uint8_t *)(&quote_body.epid_group_id))[3] = plaintext.epid_group_cert.gid.data

memcpy(&quote_body.basename, &basename, sizeof(quote_body.basename));



// 10. QE Report 생성

se_ret = sgx_create_report(NULL, NULL, &qe_report);

if(SGX_SUCCESS != se_ret) {

ret = QE_PARAMETER_ERROR;

goto CLEANUP;

}

quote_body.qe_svn = qe_report.body.isv_svn;



// 11. 대상 Enclave Report를 Quote에 복사

memcpy(&quote_body.report_body, &(p_enclave_report->body),

sizeof(quote_body.report_body));

quote_body.signature_len = (uint32_t)(sizeof(se_wrap_key_t) +

QUOTE_IV_SIZE +

sizeof(uint32_t) +
```

```
sign_size +
sizeof(sgx_mac_t));


// 12. EPID 서명 생성
ret = qe_epid_sign(p_epid_context,
plaintext,
&basename,
emp_sig_rl ? ((const  se_sig_rl_t *)emp_sig_rl)->sig_rl.bk : NULL,
&sig_rl_header,
&ec_signature,
p_enclave_report,
p_nonce,
p_qe_report,
emp_quote,
&quote_body,
(uint32_t)sign_size);
if(AE_SUCCESS != ret) {
memset_verw_s(emp_quote + sizeof(sgx_quote_t),
quote_size - sizeof(sgx_quote_t),
0, quote_size - sizeof(sgx_quote_t));
goto CLEANUP;
}
```

```
// 13. Quote 본문을 출력 버퍼에 복사

memcpy_t2u(emp_quote, &quote_body, sizeof(sgx_quote_t));



CLEANUP:

// 메모리 정리

if(p_epid_context) {

EpidMemberDelete(&p_epid_context);

}

return ret;

}
```

3.3 EPID 서명 과정 (qe_epid_sign 함수)

EPID 서명 과정은 다음과 같은 단계로 구성됩니다:

1. **EPID 멤버 컨텍스트 준비**
2. **서명할 데이터 해시 계산**
3. **EPID 서명 생성**
4. **서명 데이터 암호화 및 MAC 생성**

```
// EPID 서명 생성의 핵심 부분

EpidStatus epid_ret = EpidSign(p_epid_context,

(uint8_t*)&quote_body,

QE_QUOTE_BODY_SIZE,

(uint8_t*)&basename,
```

```
sizeof(basename),

p_sig_rl,

sig_rl_size,

(EpidSignature*)signature_buffer,

signature_size);
```

4. Intel Attestation Service 연동 및 네트워크 통신

4.1 Remote Attestation 샘플 구현

파일 위치: `SampleCode/RemoteAttestation/service_provider/ias_ra.cpp`

Intel Attestation Service와의 통신은 주로 샘플 코드에서 구현되어 있으며, 실제 프로덕션 환경에서는 서비스 제공자가 직접 구현해야 합니다.

```
// Intel Attestation Service 시뮬레이션 함수

int  ias_verify_attestation_evidence(

uint8_t*  p_isv_quote,

uint32_t  quote_size,

const  sample_ra_att_result_msg_t*  p_att_result_msg,

uint32_t  verification_result)

{

int ret = 0;

sample_ra_att_result_msg_t* p_att_result_msg_full = NULL;

int32_t verify_index = 0;

int32_t result_size = 0;
```

```
// 1. Quote 검증
if(NULL == p_isv_quote) {
return -1;
}

// 2. 플랫폼 정보 블롭 생성
sample_platform_info_blob_t* p_platform_info_blob = NULL;
ret = create_platform_info_blob(&p_platform_info_blob);
if(0 != ret) {
return ret;
}

// 3. 검증 결과 메시지 구성
result_size = sizeof(sample_ra_att_result_msg_t) +
sizeof(sample_platform_info_blob_t);
p_att_result_msg_full = (sample_ra_att_result_msg_t*)malloc(result_size);
if(NULL == p_att_result_msg_full) {
ret = -1;
goto CLEANUP;
}

memset(p_att_result_msg_full, 0, result_size);
```

```
p_att_result_msg_full->platform_info_blob = *p_platform_info_blob;

// 4. MAC 계산 (플랫폼 정보 블롭에 대한)

ret = sample_rijndael128GCM_encrypt(

&g_rk_priv_key,

(uint8_t*)&p_att_result_msg_full->platform_info_blob,

sizeof(sample_platform_info_blob_t),

(uint8_t*)&p_att_result_msg_full->mac,

sizeof(sample_mac_t),

NULL, 0);



CLEANUP:

if(p_platform_info_blob) {

free(p_platform_info_blob);

}

if(p_att_result_msg_full) {

free(p_att_result_msg_full);

}

return ret;

}
```

4.2 네트워크 통신 구현

파일 위치: `SampleCode/RemoteAttestation/service_provider/network_ra.cpp`

```
// HTTP 기반 네트워크 통신 함수
int ra_network_send_receive(const char* server_url,
const ra_samp_request_header_t* p_req,
ra_samp_response_header_t** p_resp)
{
int ret = 0;
CURL* curl_handle = NULL;
CURLcode curl_ret = CURLE_OK;
struct curl_slist* headers = NULL;
string response_buffer;
string header_buffer;
long response_code = 0;

// 1. cURL 초기화
curl_handle = curl_easy_init();
if(NULL == curl_handle) {
ret = -1;
goto CLEANUP;
}

// 2. HTTP 헤더 설정
headers = curl_slist_append(headers, "Content-Type: application/json");
```

```
    headers = curl_slist_append(headers, "Ocp-Apim-Subscription-Key: [YOUR_SUBSCRIPT

    curl_easy_setopt(curl_handle, CURLOPT_HTTPHEADER, headers);

    curl_easy_setopt(curl_handle, CURLOPT_URL, server_url);

    curl_easy_setopt(curl_handle, CURLOPT_POST, 1L);



    // 3. 요청 데이터 설정

    string request_body = create_json_request(p_req);

    curl_easy_setopt(curl_handle, CURLOPT_POSTFIELDS, request_body.c_str());

    curl_easy_setopt(curl_handle, CURLOPT_POSTFIELDSIZE, request_body.length());



    // 4. 응답 콜백 설정

    curl_easy_setopt(curl_handle, CURLOPT_WRITEFUNCTION, write_callback);

    curl_easy_setopt(curl_handle, CURLOPT_WRITEDATA, &response_buffer);

    curl_easy_setopt(curl_handle, CURLOPT_HEADERFUNCTION, header_callback);

    curl_easy_setopt(curl_handle, CURLOPT_HEADERDATA, &header_buffer);



    // 5. SSL 설정

    curl_easy_setopt(curl_handle, CURLOPT_SSL_VERIFYPEER, 1L);

    curl_easy_setopt(curl_handle, CURLOPT_SSL_VERIFYHOST, 2L);



    // 6. HTTP 요청 실행

    curl_ret = curl_easy_perform(curl_handle);

    if(CURLE_OK != curl_ret) {
```

```
ret = -1;

goto CLEANUP;

}



// 7. 응답 코드 확인

curl_easy_getinfo(curl_handle, CURLINFO_RESPONSE_CODE, &response_code);

if(200 != response_code) {

ret = -1;

goto CLEANUP;

}



// 8. 응답 파싱

ret = parse_response_header(header_buffer.c_str(), p_resp);

if(0 != ret) {

goto CLEANUP;

}



ret = parse_response_body(response_buffer.c_str(), *p_resp);



CLEANUP:

if(headers) {

curl_slist_free_all(headers);

}
```

```
if(curl_handle) {
curl_easy_cleanup(curl_handle);
}
return ret;
}
```

4.3 JSON 메시지 포맷

Intel Attestation Service와의 통신에서 사용되는 JSON 메시지 포맷:

```
// Quote 검증 요청
{
"isvEnclaveQuote": "[BASE64_ENCODED_QUOTE]",
"pseManifest": "[BASE64_ENCODED_PSE_MANIFEST]",
"nonce": "[BASE64_ENCODED_NONCE]"
}

// Quote 검증 응답
{
"id": "[REQUEST_ID]",
"timestamp": "[ISO8601_TIMESTAMP]",
"version": 4,
"epidPseudonym": "[BASE64_ENCODED_EPID_PSEUDONYM]",
"isvEnclaveQuoteStatus": "OK",
```

```
"isvEnclaveQuoteBody": "[BASE64_ENCODED_QUOTE_BODY]",

"revocationReason": null,

"pseManifestStatus": "OK",

"pseManifestHash": "[BASE64_ENCODED_PSE_MANIFEST_HASH]",

"platformInfoBlob": "[BASE64_ENCODED_PLATFORM_INFO_BLOB]",

"nonce": "[BASE64_ENCODED_NONCE]",

"advisoryURL": "https://security-center.intel.com",

"advisoryIDs": ["INTEL-SA-00334", "INTEL-SA-00615"]

}
```


결론

이 분석을 통해 Intel SGX SDK의 Remote Attestation 구현 과정을 다음과 같이 정리할 수 있습니다:

1. **sgx_create_report()**: EREPORT 하드웨어 명령어를 통해 Enclave Report 생성
2. **AESM Service IPC**: Unix Domain Socket과 Protobuf를 사용한 프로세스 간 통신
3. **Quoting Enclave**: EPID 라이브러리를 사용한 Quote 생성 및 서명
4. **Intel Attestation Service**: HTTP/JSON 기반 원격 검증 서비스 연동

각 단계는 엄격한 보안 검증과 에러 처리를 포함하며, 하드웨어 보안 기능과 소프트웨어 구현이 긴밀하게 연동되어 전체 Remote Attestation 프로세스를 구현합니다.